02_skill_roster

June 29, 2026

1 02 — Skill roster

Step 2 of 3. Build the candidate universe, score each wallet from its **closed-position history**, apply the hard gates (Blueprint §3), and save a vetted roster to `roster.json`. Run this **weekly**.

This is our main defense against survivorship/luck bias: a wallet earns its place by a *long, profitable, consistent* track record — not by being hot this month.

[6]:
```
import importlib, pmc, json, time
from datetime import datetime, timezone, timedelta
importlib.reload(pmc)
from pmc import CFG, get_leaderboard, get_closed_positions, get_open_positions,␣
 ↪portfolio_value
import pandas as pd
```

1.1 1. Candidate universe — union of `all` (proven) and `month` (active)

[7]:
```
candidates = {}
for w in CFG.LEADERBOARD_WINDOWS:
    for row in get_leaderboard(window=w, limit=CFG.LEADERBOARD_TOP_N):
        candidates.setdefault(row["wallet"], row)
print(f"{len(candidates)} unique candidate wallets")
```

```
252 unique candidate wallets
```

1.2 2. Score each candidate from closed positions

For every wallet we compute the skill-gate metrics. Each resolved position is a 'trade'; `realizedPnl > 0` counts as a win. ROI = total realized PnL / total cost.

[8]:
```
def score_wallet(wallet: str) -> dict | None:
    closed = get_closed_positions(wallet)
    if not closed:
        return None
    pnls, costs, last_ts = [], [], 0
    for p in closed:
        rp = float(p.get("realizedPnl") or p.get("cashPnl") or 0)
        cost = float(p.get("totalBought") or p.get("initialValue") or 0)
        pnls.append(rp); costs.append(abs(cost))
```

```
        ts = p.get("endDate") or p.get("timestamp") or 0
        if isinstance(ts, (int, float)):
            last_ts = max(last_ts, ts)
    n = len(pnls)
    total_pnl = sum(pnls)
    total_cost = sum(costs) or 1.0
    wins = sum(1 for x in pnls if x > 0)
    win_rate = wins / n
    roi = total_pnl / total_cost
    best_share = (max(pnls) / total_pnl) if total_pnl > 0 else 1.0
    return {
        "wallet": wallet,
        "resolved_trades": n,
        "lifetime_pnl": round(total_pnl, 2),
        "win_rate": round(win_rate, 3),
        "roi": round(roi, 3),
        "best_trade_share": round(best_share, 3),
        "n_positions": n,
    }

scored = []
for i, w in enumerate(candidates):
    s = score_wallet(w)
    if s:
        # use the leaderboard's authoritative lifetime PnL (our sample is␣
 ↪capped/recent)
        s["lifetime_pnl"] = round(float(candidates[w].get("pnl") or␣
 ↪s["lifetime_pnl"]), 2)
        scored.append(s)
    if (i+1) % 20 == 0: print(f"  scored {i+1}/{len(candidates)}")
sdf = pd.DataFrame(scored)
print(f"scored {len(sdf)} wallets")
sdf.head()
```

```
  scored 20/252
  scored 40/252
  scored 60/252
  scored 80/252
  scored 100/252
  scored 120/252
  scored 140/252
  scored 160/252
  scored 180/252
  scored 200/252
  scored 220/252
  scored 240/252
scored 250 wallets
```

[8]:

	wallet	resolved_trades	lifetime_pnl	win_rate	roi	best_trade_share	n_positions
0	0x56687bf447db6ffa42ffe2204a05edaa20f55839	22	22053933.75	0.818	0.513	0.376	22
1	0x1f2dd6d473f3e824cd2f8a89d9c69fb96f6ad0cf	66	16619506.63	0.606	0.219	0.584	66
2	0x204f72f35326db932158cba6adff0b9a1da95e14	300	14637655.01	0.160	-0.056	1.000	300
3	0x6a72f61820b26b1fe4d956e17b6dc2a1ea3033ee	300	11386689.78	0.547	0.087	0.250	300
4	0x2005d16a84ceefa912d4e380cd32e7ff827875ea	300	10239768.27	0.000	0.000	1.000	300

1.3 3. Apply the hard gates (Blueprint §3)

[9]:
```
def passes_gate(r) -> bool:
    return (
        r["resolved_trades"] >= CFG.MIN_RESOLVED_TRADES and
        r["lifetime_pnl"]   >  CFG.MIN_LIFETIME_PNL and
        r["win_rate"]       >= CFG.MIN_WIN_RATE and
        r["roi"]            >= CFG.MIN_ROI and
        r["best_trade_share"] <= CFG.MAX_SINGLE_TRADE_PROFIT_SHARE and
        r["n_positions"]    <= CFG.MARKET_MAKER_POSITION_COUNT   # exclude␣
 ↪market-makers
    )

roster = sdf[sdf.apply(passes_gate, axis=1)].copy() if len(sdf) else sdf
# skill score: normalize a blend of win_rate, roi, log(pnl) to 0..1
if len(roster):
    import numpy as np
    z = lambda s: (s - s.min()) / ((s.max() - s.min()) or 1)
    roster["skill"] = (0.4*z(roster["win_rate"]) + 0.4*z(roster["roi"]) +
                       0.2*z(np.log1p(roster["lifetime_pnl"].clip(lower=1)))).
 ↪round(3)
    roster = roster.sort_values("skill", ascending=False).head(CFG.
 ↪TARGET_ROSTER_SIZE)
print(f"roster size: {len(roster)}")
roster
```

```
roster size: 27
```

[9]:

	wallet	resolved_trades
48	0xd38b71f3e8ed1af71983e5c309eac3dfa9b35029	287
28	0xdb27bf2ac5d428a9c63dbc914611036855a6c56e	300
40	0x17db3fcd93ba12d38382a0cade24b200185c5f6d	97
136	0x9b3dcd99eec7fe11602e6534e6302c0f318d7422	110

25	0xdc876e6873772d38716fda7f2452a78d426d7ab6	300
60	0x7fb7ad0d194d7123e711e7db6c9d418fac14e33d	300
95	0xde7be6d489bce070a959e0cb813128ae659b5f4b	190
43	0x5966db1fe50763c9e3c014d756369bad07e1f804	115
79	0xea2b4224411e723499a803ce3f4758779fb31fc6	300
116	0x51393c00184b39182f09a8a62b8549642e69a8db	300
160	0xc8075693f48668a264b9fa313b47f52712fcc12b	150
90	0xf2f6af4f27ec2dcf4072095ab804016e14cd5817	300
178	0xe16d3f2a5807999b358affd9445c3a09e45e5e30	300
33	0x16b29c50f2439faf627209b2ac0c7bbddaa8a881	300
91	0xbddf61af533ff524d27154e589d2d7a81510c684	300
154	0x4bff30af91642dc7d2b19a8664378fe55c45fc26	186
163	0x2a69660046d7acc4ab204d7cc5ba78b0776cd2f7	300
157	0xc6587b11a2209e46dfe3928b31c5514a8e33b784	300
242	0xe015b5a2a299167be835a2fd1e86f09c49e06ffd	300
138	0x2b3ff45c91540e46fae1e0c72f61f4b049453446	112
93	0x26437896ed9dfeb2f69765edcafe8fdceaab39ae	192
244	0x3c593aeb73ebdadbc9ce76d4264a6a2af4011766	300
124	0x1cc16713196d456f86fa9c7387dd326a7f73b8df	300
88	0xb91aeb5accc33a5f9a8615b8ed6b2d352e913987	138
201	0xfc0847fe2ea938d6e2624650eb29ef8d8142a137	300
233	0x7d787c72c8317046fda90814a93c87fc192529a5	300
146	0xb74711992caf6d04fa55eecc46b8efc95311b050	250

	lifetime_pnl	win_rate	roi	best_trade_share	n_positions	skill
48	2673262.16	0.976	0.513	0.035	287	0.965
28	4055592.54	0.950	0.458	0.061	300	0.918
40	3133968.20	0.959	0.248	0.228	97	0.729
136	1069068.64	0.891	0.403	0.043	110	0.720
25	4526176.05	0.717	0.321	0.277	300	0.562
60	2282134.46	0.733	0.279	0.366	300	0.497
95	1535290.64	0.774	0.255	0.304	190	0.493
43	3044108.69	0.730	0.239	0.089	115	0.478
79	1777311.83	0.673	0.269	0.092	300	0.409
116	1263905.09	0.737	0.200	0.184	300	0.394
160	562740.40	0.700	0.271	0.071	150	0.362
90	1562750.05	0.773	0.099	0.061	300	0.359
178	456327.47	0.607	0.396	0.040	300	0.358
33	3648171.81	0.713	0.072	0.319	300	0.329
91	1562452.99	0.740	0.101	0.055	300	0.326
154	647096.46	0.683	0.206	0.043	186	0.298
163	554093.31	0.700	0.174	0.184	300	0.278
157	601973.69	0.790	0.049	0.170	300	0.270
242	227118.50	0.697	0.226	0.089	300	0.260
138	1044505.33	0.679	0.122	0.230	112	0.253
93	1538290.80	0.646	0.125	0.178	192	0.246
244	223774.58	0.673	0.206	0.183	300	0.216

```
124    1163914.38     0.640  0.102            0.291          300  0.202
88     1633834.11     0.609  0.105            0.087          138  0.194
201     342986.87     0.703  0.051            0.163          300  0.143
233     240323.24     0.687  0.091            0.285          300  0.137
146     992969.73     0.596  0.069            0.369          250  0.116
```

1.4 4. Save the roster (consumed by notebook 03)

[10]:
```
payload = {
    "generated_at": datetime.now(timezone.utc).isoformat(),
    "config": {k: getattr(CFG, k) for k in␣
 ↪["MIN_RESOLVED_TRADES","MIN_LIFETIME_PNL","MIN_WIN_RATE","MIN_ROI"]},
    "wallets": roster.to_dict("records") if len(roster) else [],
}
with open("roster.json", "w") as f:
    json.dump(payload, f, indent=2, default=str)
print(f"saved roster.json with {len(payload['wallets'])} wallets")
```

```
saved roster.json with 27 wallets
```

Tuning note. If the roster comes back empty, your gates are too strict for the sample the API returned — loosen `MIN_RESOLVED_TRADES` / `MIN_LIFETIME_PNL` in `pmc.py` and re-run. The backtest (future notebook) is what sets these properly.

Next: `03_consensus_signals.ipynb`.